UNIVERSITY OF MAKATI
J.P. Rizal Extension, West Rembo, Makati City

Algorithm Architecture Specification and Software Engineering Proposal

Metadata Field	Project Team Specification
Project Title	Oak Sports Academy Training Registration and Management System Enhancement
Team Name	Oak Sports - Group 5
Section & Year	III-BCSAD
Team Members & Roles	Lead Algorithm Architect: Nathaniel Babalcon Backend / Systems Engineer: Hexel John Ventura, Rommel Villanueva Data / QA Engineer: Jurie Aplaca Systems Analyst / Technical Lead: Aaron Abiera
Repository URL	https://github.com/hex-xel/oak-sports-academy
Submission Date	10/23/2026

Course: ALGOCOM (Algorithms and Complexity) / SOFTENG (Software Engineering)

Summary

This proposal models weekly training schedule selection as a preference-weighted interval scheduling problem. Given available sessions, participant rankings, and a requested frequency of two or three sessions per week, a Dynamic Programming algorithm returns a highest-scoring set of non-overlapping sessions or reports that no feasible schedule exists. The algorithm is implemented as a modular component of the existing backend and does not access the database or user-interface logic directly. This report presents the mathematical model, algorithm, complexity, service interface, implementation backlog, and academic defense.

Table of Contents

Task 1 - Computational Problem Modeling **3**
- Problem statement and domain context 3
- Formal mathematical specification 3
- Decision variables and objective 3
- Inputs, constraints, and output certificate 4
- Boundary conditions and assumptions 4

Task 2 - Algorithmic Paradigm and Logic **5**
- Selection and justification 5
- Recurrence 6
- Pseudocode 6
- Initialization and tie-breaking 6

Task 3 - Theoretical Complexity and Scalability **7**

Task 4 - Decoupled Architecture and Data Contract **8**
- POST /api/v1/algorithm/solve request 9
- Response certificate 9

Task 5 - Agile Backlog and Empirical Validation **10**
- User stories 10
- Engineering implementation ticket 10
- Empirical validation plan 11
- Benchmark procedure 11

Task 6 - Academic Review and Defense **11**
- 1. Why must heavy algorithmic processing be decoupled from database ORMs and web request threads? 11
- 2. If your proposed algorithm has an exponential worst-case complexity, what heuristic, pruning, or approximation technique will maintain production viability? 12
- 3. What is the operational difference between an Exact Algorithm and an Approximate / Heuristic Algorithm in production optimization? 12
- 4. How do you distinguish between an empirical benchmarking profiling approach (post-implementation) and an asymptotic theoretical Big-O analysis (pre-implementation)? 12
- 5. If your project utilizes machine learning or user-preference matrices, how will you mitigate the Cold Start Problem or Data Sparsity Problem during initial launch? 12
- 6. How does your algorithm handle edge-case inputs (e.g., disconnected subgraphs, singular matrices, cyclic dependencies) without crashing the host service? 13

References **13**

Presentation Deck **15**

Task 1 - Computational Problem Modeling

Problem statement and domain context

Oak Sports Academy participants select a training frequency and available schedules. When several weekly sessions are available, finding a set that meets the requested frequency, avoids time conflicts, and best matches participant preferences is a scheduling optimization problem. Research on course-seat allocation similarly considers participant preferences and limited schedule options (Bichler & Merting, 2021).

The project requirements allow participants to select a training frequency of twice or thrice per week and an available schedule. This proposal adds a recommendation that identifies a preferred feasible combination while leaving final choice and Academy review with the participant and staff.

Formal mathematical specification

Let S = {s1, ..., sn} be the available candidate weekly sessions, sorted by end time. Each session s_i has start time a_i, end time b_i, and preference score w_i. Let $q \in \{2, 3\}$ be the requested number of sessions per week. Let p(i) be the largest index less than i whose session ends at or before a_i, or 0 if none exists.

<!-- AI Assistance: ChatGPT was used to help clarify notation for the scheduling model and to review the presentation of constraints. The team selected the problem formulation, checked the assumptions against the project requirements, and verified the final mathematical model and citations. -->

Decision variables and objective

For each session i, $x_i = 1$ when selected and 0 otherwise. Maximize $\Sigma(i=1..n)\ w_i\ x_i$, subject to $\Sigma(i=1..n)\ x_i = q$ and $x_i + x_j \leq 1$ for every pair of overlapping sessions. Back-to-back sessions are compatible when one ends at or before the other begins.

$$max \sum_{i=1}^{n} w_i x_i$$

$$\sum_{i=1}^{n} x_i = q, x_i \in \{0,1\}$$

$$x_i + x_j \leq 1$$

Inputs, constraints, and output certificate

Element	Specification
Input parameters (I)	Available candidate sessions; session IDs; recurring weekly start/end times; participant preference scores; requested weekly frequency q.
Boundary conditions & constraints (C)	q must be 2 or 3; selected sessions must be available, unique, and pairwise non-overlapping; exactly q sessions must be selected.
Objective function f(x)	Optimization problem (maximization): maximize $f(x) = \Sigma\, w_i\, x_i$ over selections $x_i \in \{0, 1\}$, subject to exactly q selected sessions and no overlapping pair.
Assumptions	Weekly recurring sessions are represented consistently on a single week timeline and timezone. First version checks participant schedule conflicts only; instructor conflicts and class capacity are excluded.
Output certificate (O)	SUCCESS with exactly q selected session IDs and total preference score, or NO_FEASIBLE_SCHEDULE. Include execution metrics.

Preference scores: derive integer scores from the participant ranking (for example, first choice = 3, second = 2, third = 1). If rankings are absent, define and document a neutral fallback before implementation.

Boundary conditions and assumptions

The requested frequency is $q \in \{2,3\}$. Only available candidate sessions with valid start and end times are considered. Each session must have a positive duration, and all times must use the Academy's defined timezone. Sessions that end at the same time another begins are considered compatible. The algorithm selects exactly q sessions; if no such combination exists, it returns NO_FEASIBLE_SCHEDULE.

Each preference score is an integer from 1 to 3, where 3 represents the participant's highest preference and 1 the lowest. If two or more schedule combinations have the same total score, the algorithm uses a deterministic tie-break: choose the combination whose sorted session IDs come first alphabetically. If the participant provides no ranking, all available sessions receive the same score.

This model checks conflicts within one participant's weekly schedule. It does not assign instructors, manage class capacity, or resolve conflicts between different participants.

Task 2 - Algorithmic Paradigm and Logic

Selection and justification

Algorithm: Preference-Weighted Weekly Schedule Selection. Paradigm: Dynamic Programming using weighted interval scheduling with an additional session-count dimension. The problem has optimal substructure: the best schedule either skips a candidate session or includes it and combines it with the best compatible earlier schedule. Dynamic Programming is an approved PBL paradigm (Cormen et al., 2022).

Schema field	Engineering specification
Algorithm name	Preference-Weighted Weekly Schedule Selection
Design paradigm	Dynamic Programming / weighted interval scheduling
Foundational reference	Cormen, Leiserson, Rivest, and Stein (2022), Introduction to Algorithms, 4th ed.
Domain reference	Bichler and Merting (2021), preference-aware course-seat allocation.
Baseline alternative	Exhaustively enumerate all q-session combinations and reject overlaps; conceptual baseline only.
Computational justification	Stores subproblem solutions rather than enumerating all combinations.
Invariant	DP[i,j] is the highest preference score using exactly j compatible sessions from the first i sorted candidates.

Recurrence

Base cases: DP[i,0] = 0 for all i; DP[0,j] = −∞ for j > 0. For i = 1..n and j = 1..q:

DP[i,j] = max(DP[i−1,j], wi + DP[p(i),j−1]).

The first term skips session i. The second selects it and adds its score to the best compatible schedule ending before it. Store the winning choice to reconstruct sessions. If DP[n,q] = −∞, return NO_FEASIBLE_SCHEDULE.

Pseudocode

```
FUNCTION SelectWeeklySchedule(sessions, q):
    REQUIRE q is 2 or 3
    available ← sessions where available = true
    SORT available by end time
    n ← length(available)
    FOR i FROM 1 TO n:
        p[i] ← latest index < i whose end ≤ start[i], or 0
    FOR i FROM 0 TO n: DP[i][0] ← 0
    FOR j FROM 1 TO q: DP[0][j] ← −∞
    FOR i FROM 1 TO n:
        FOR j FROM 1 TO q:
            skip ← DP[i−1][j]
            include ← preference[i] + DP[p[i]][j−1] if feasible else −∞
            DP[i][j] ← max(skip, include)
            choice[i][j] ← SELECT if include > skip; SKIP if skip > include; otherwise apply the deterministic
tie-break rule
    IF DP[n][q] = −∞: RETURN NO_FEASIBLE_SCHEDULE
    Reconstruct selected sessions by following choice from (n,q)
    RETURN selected sessions, DP[n][q], SUCCESS
```

<!-- AI Assistance: ChatGPT was used as a brainstorming and review aid for expressing the Dynamic Programming recurrence and pseudocode clearly. The team chose the algorithmic approach, reviewed each step, and verified that the logic, tie-breaking rule, and result reconstruction match the stated model. -->

Initialization and tie-breaking

Initialize every DP table cell to an infeasible value before filling the table. Set $DP[i,0] = 0$ for all i, and $DP[0,j] = -\infty$ for every $j > 0$. If there are no available sessions, return NO_FEASIBLE_SCHEDULE.

When the include and skip choices have equal scores, use the specified deterministic tie-break rule. Sort sessions by end time, then start time, then session ID. Preserve enough choice information to reconstruct the selected sessions.

Task 3 - Theoretical Complexity and Scalability

Let n be the number of candidate sessions and $q \in \{2,3\}$ the number requested.

Sorting by end time takes $O(n \log n)$. Computing the compatible predecessor for each session using binary search takes $n \cdot O(\log n) = O(n \log n)$. Filling the dynamic programming table takes nq constant-time updates, or $O(nq)$. Reconstructing the result takes at most $O(n + q)$.

Therefore,

$$T(n, q) = O(n \log n) + O(n \log n) + O(nq) + O(n + q)$$

which simplifies to

$$T(n, q) = O(n \log n + nq).$$

Because $q \leq 3$, q is bounded by a constant for this application, so the total time simplifies to $O(n \log n)$. Using a comparison sort with $O(n \log n)$ worst-case time, the best, average, and worst asymptotic cases remain $O(n \log n + nq)$, because the algorithm sorts the candidates and fills the DP table for every request.

The score and choice tables require $O(nq)$ space. The session list and predecessor indexes require $O(n)$. Total auxiliary space is $O(nq + n)$, which becomes $O(n)$ when $q \leq 3$. The Dynamic Programming analysis is based on the subproblem and recurrence approach described by Cormen et al. (2022).

Dimension	Engineering parameter	Scalability impact and mitigation
Volume	Candidate sessions n; prototype sizes 100, 1,000, 5,000	Sorting and predecessor calculation grow as O(n log n). Limit inputs to available sessions relevant to the participant.
Velocity	Recommendation requests per second	Requests are independent. Keep the algorithm stateless; benchmark throughput before introducing a queue or worker pool.
Variety	IDs, availability, recurring day/time intervals, preference scores	Validate and normalize data and timezone before solving. Reject malformed intervals.

Benchmark sizes and expected traffic are planning assumptions, not measured Oak Sports Academy production data.

<!-- AI Assistance: ChatGPT was used to review the explanation of time and space complexity. The team independently checked the derivation, including sorting, predecessor computation, Dynamic Programming table construction, and the

simplification for q in {2, 3}. -->

Task 4 - Decoupled Architecture and Data Contract

The algorithm accepts validated session data and returns selected session IDs and a result certificate. It does not query the database or format UI output. Because the proposed algorithm is deterministic Dynamic Programming rather than a machine learning model, it has no trained model to deploy, retrain, or version and needs no specialized hardware, so it does not need to be isolated as a separate service. For the initial student implementation, implement it as a TypeScript module inside the existing Node.js/Express backend; consider a worker process only if measured CPU load warrants it. The backend remains responsible for authorization and data retrieval (Oak Sports Academy, 2025).

Component flow: Web/Mobile Client → Express API and Orchestration → PostgreSQL via Prisma (authorized session retrieval); Express passes validated JSON to the Schedule DP Module, which runs inside the same backend application; the module returns a certificate through the API to the client.

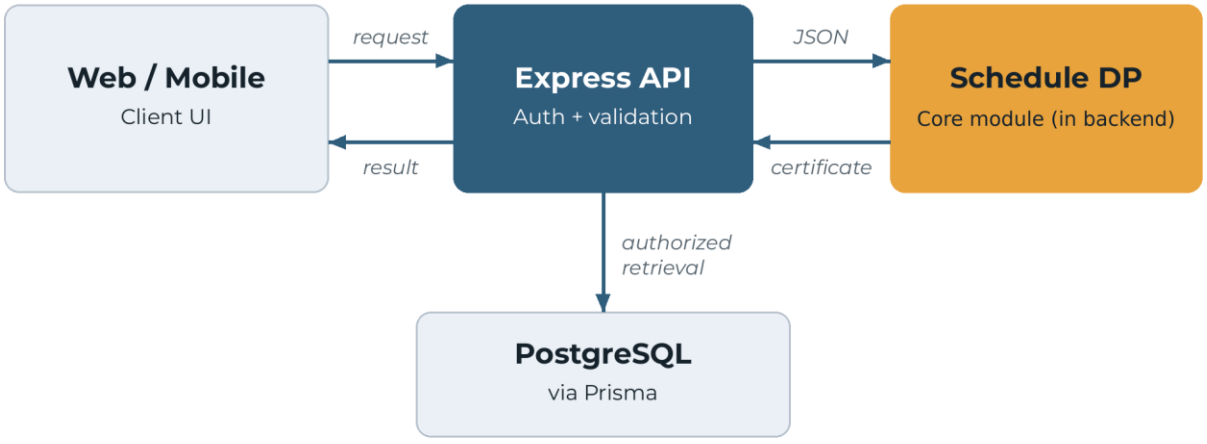


Figure 1. *Decoupled component architecture. The Express API orchestrates; the Schedule DP Module never touches the database or UI.*

POST /api/v1/algorithm/solve request

```
{
  "requestId": "req-123",
  "participantId": "participant-456",
  "sessionsPerWeek": 2,
  "sessions": [
    {"sessionId":"mon-1700","start":"2026-11-02T17:00:00+08:00",
     "end":"2026-11-02T18:00:00+08:00","preferenceScore":3,"available":true},
    {"sessionId":"wed-1700","start":"2026-11-04T17:00:00+08:00",
     "end":"2026-11-04T18:00:00+08:00","preferenceScore":2,"available":true}
  ]
}
```

Response certificate

```
{
  "requestId":"req-123", "status":"SUCCESS",
  "selectedSessionIds":["mon-1700","wed-1700"],
  "totalPreferenceScore":5, "sessionsRequested":2, "sessionsSelected":2,
  "executionMetrics":{"candidateCount":2,"runtimeMs":1.42}
}
```

The API requires requestId, participantId, sessionsPerWeek, and sessions. sessionsPerWeek must be the integer 2 or 3. Each session requires a unique sessionId, a valid start and end timestamp with timezone, a preferenceScore integer from 1 to 3, and an available Boolean. The end time must be later than the start time.

Return HTTP 200 with SUCCESS and the selected schedule when a feasible combination exists. Return HTTP 200 with NO_FEASIBLE_SCHEDULE and an empty selection when the input is valid but no combination exists. Return HTTP 400 for invalid input and HTTP 500 with a generic error identifier for unexpected failures (Fielding et al., 2022). Do not include private participant data in error messages or result certificates.

The request and response schemas should be documented in OpenAPI 3.1. (OpenAPI Initiative, 2021).

<!-- AI Assistance: ChatGPT was used to help organize the explanation of service boundaries and example request and response schemas. The team defined the architecture, validated the separation of the algorithm from database and UI concerns, and reviewed the contract against the project requirements and cited standards. -->

Task 5 - Agile Backlog and Empirical Validation

User stories

1. As a Participant, I want the system to recommend available weekly sessions that meet my requested frequency and avoid time conflicts, so that I can choose a feasible schedule that fits my preferences.
2. As Academy Staff, I want to review the recommended session set and its feasibility result, so that I can verify schedule options before processing enrollment.

Engineering implementation ticket

Field	Specification
Ticket ID	DEV-ALGO-101
Module / Service	apps/api/src/modules/schedules/algorithm/ (proposed; align with repository layout)
Associated algorithm	Preference-Weighted Weekly Schedule Selection using Dynamic Programming
Edge cases and security	Reject q outside {2,3}, duplicate IDs, malformed or zero-length intervals, invalid scores and missing fields. Ignore unavailable sessions. Return no-feasible if fewer than q compatible sessions exist. Authorize participant before retrieving sessions. Do not expose private data.
Implementation steps	1. Define request/response types. 2. Validate and normalize sessions. 3. Sort by end time. 4. Compute p(i). 5. Fill score and choice tables. 6. Reconstruct solution. 7. Return certificate and metrics.
Correctness assertions	Successful output has exactly q unique available sessions; no selected pair overlaps; returned score equals sum of selected scores; fixtures match hand-computed outcomes.
Benchmark assertion	On fixed CI runner, deterministic 5,000-session input and q=3: algorithm runtime <250 ms. Record runtime and environment. This is a proposed target to validate, not a measured result.

Empirical validation plan

- Generate deterministic synthetic weekly sessions at n = 100, 1,000, and 5,000, including feasible, infeasible, back-to-back, and overlapping cases.
- Run correctness tests separately from timing measurements; compare small cases against manually verified expected schedules.
- Record runtime, input size, runtime version, machine/CI runner, and whether preprocessing is included.
- Do not claim the 250 ms goal until measured. Compare growth trends with the theoretical O(n log n + nq) analysis.

Benchmark procedure

Run each input size at least 10 times after one warm-up run. Record the median runtime, input size, Node.js version, operating system, and CI runner or computer model. Include preprocessing in the measurement when measuring the end-to-end algorithm, and state clearly if any measurement excludes sorting or predecessor computation. Keep the input generator deterministic so that repeated runs use the same sessions. The 250 ms threshold is a target for the specified environment; revise it if initial measurements show it is not realistic, and document the measured results separately from the theoretical analysis.

<!-- AI Assistance and Verification: ChatGPT was used to suggest how to present a reproducible benchmarking plan. The team selected the test sizes, validation cases, measurement fields, and performance target; all reported benchmark results will be based on the team's own implementation and measurements. -->

Task 6 - Academic Review and Defense

1. Why must heavy algorithmic processing be decoupled from database ORMs and web request threads?

The algorithm should receive validated data and return a result without querying the database or formatting UI output. This separation of concerns improves testability and keeps persistence changes separate from mathematical logic. It does not require a separate service, however: the proposed algorithm is deterministic Dynamic Programming, not a machine learning model, so it has no model to train, serve, or version and needs no specialized hardware. Because it is small, O(n log n+nq), it can run as a module inside the backend, and a separate worker is unnecessary unless

benchmarking shows CPU work delaying requests. Node.js worker threads are intended for CPU-intensive JavaScript operations (Node.js, n.d.).

2. If your proposed algorithm has an exponential worst-case complexity, what heuristic, pruning, or approximation technique will maintain production viability?

The proposed DP is polynomial, not exponential: O(n log n+nq), with q≤3. It avoids enumerating all candidate combinations by reusing subproblem results (Cormen et al., 2022). If future constraints invalidate the recurrence, reformulate and reanalyze before adding heuristics.

3. What is the operational difference between an Exact Algorithm and an Approximate / Heuristic Algorithm in production optimization?

The proposed Dynamic Programming algorithm is exact for the defined model: given the candidate sessions, scores, and requested frequency, it finds a feasible set of exactly q sessions with the highest total score. This optimality claim applies only to the stated constraints. A heuristic may return a feasible schedule faster for a more complex problem, but it does not guarantee the highest possible score unless it also provides an optimality guarantee (Cormen et al., 2022).

4. How do you distinguish between an empirical benchmarking profiling approach (post-implementation) and an asymptotic theoretical Big-O analysis (pre-implementation)?

Asymptotic analysis describes how resource use grows as input size increases under the algorithm's mathematical model. Empirical benchmarking measures the implemented program on specific hardware, runtime versions, and test data. The $O(n\ log\ n + nq)$ analysis predicts growth; the 5,000-session runtime assertion checks actual performance in a selected environment. Neither result substitutes for the other (Cormen et al., 2022).

5. If your project utilizes machine learning or user-preference matrices, how will you mitigate the Cold Start Problem or Data Sparsity Problem during initial launch?

This proposal does not use machine learning or a user-preference matrix. It uses explicit session rankings submitted by the participant. If a ranking is missing, the system will assign equal scores to available sessions or ask the participant to provide preferences; the team must choose and document one behavior. This avoids claiming to infer preferences from data the system does not have.

6. How does your algorithm handle edge-case inputs (e.g., disconnected subgraphs, singular matrices, cyclic dependencies) without crashing the host service?

The API validates the request before running the algorithm. Invalid frequency, duplicate session IDs, missing fields, malformed timestamps, non-positive session durations, or invalid preference scores produce a controlled validation response. Valid input with fewer than q compatible sessions returns NO_FEASIBLE_SCHEDULE. Back-to-back sessions are compatible. Recurring sessions are normalized to the Academy timezone and a common weekly timeline before comparison. The API catches unexpected errors and returns a generic error identifier without exposing stack traces or participant data. API data types and constraints are documented in the OpenAPI schema (OpenAPI Initiative, 2021).

References

- Bichler, M., & Merting, S. (2021). Randomized scheduling mechanisms: Assigning course seats in a fair and efficient way. *Production and Operations Management*, *30*(10), 3540–3559. https://doi.org/10.1111/poms.13449
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.) [Online]. The MIT Press. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/
- Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc9110
- Node.js. (n.d.). *Worker threads | Node.js v26.12.1 documentation*. Retrieved October 23, 2026, from https://nodejs.org/api/worker_threads.html
- Oak Sports Academy. (2025). *User stories* (Version 1, August 27, 2025) [Project document].
- OpenAPI Initiative. (2021, February 15). *OpenAPI specification* (Version 3.1.0). Retrieved October 23, 2026, from https://spec.openapis.org/oas/v3.1.0.html